

Cloud Computing : Assignment 1

Neel Prajapati / 202611022

August 12, 2026

Exercise 1: TCP Client-Server Q&A System

Problem Statement

Create a TCP server and client using the sockets library. Make them communicate with each other by making a Q&A system between them. Make a list of 4-5 Q&A pairs in a file and use these for the application.

Input Files

1. qa.txt

This file contains the predefined questions and answers separated by a pipe (|) symbol.

```
1 What is TCP?|TCP is a connection-oriented and reliable transport layer
  protocol.
2 What is UDP?|UDP is a connectionless and unreliable transport layer
  protocol.
3 What is a socket?|A socket is an endpoint used for communication
  between two processes.
4 What is IP?|IP stands for Internet Protocol and is used for addressing
  and routing packets.
5 What is HTTP?|HTTP stands for HyperText Transfer Protocol and is used
  for communication on the web.
```

2. TCP Server (server.py)

```
1 import socket
2
3 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4
5 server.bind(("127.0.0.1", 8080))
6 server.listen(1)
7
8 print("TCP Server started...")
9 print("Waiting for client...")
10
11 client, address = server.accept()
12
13 print("Client connected!")
14
15 while True:
```

```

16 question = client.recv(1024).decode()
17
18 if not question:
19     break
20
21 if question == "exit":
22     break
23
24 answer = "Question not found."
25
26 file = open("qa.txt", "r")
27
28 for line in file:
29     parts = line.strip().split("|", 1)
30
31     if len(parts) == 2:
32         storedQuestion = parts[0]
33         storedAnswer = parts[1]
34
35         if question == storedQuestion:
36             answer = storedAnswer
37             break
38
39 file.close()
40
41 print("Question:", question)
42 print("Answer:", answer)
43
44 client.send(answer.encode())
45
46 client.close()
47 server.close()

```

3. TCP Client (client.py)

```

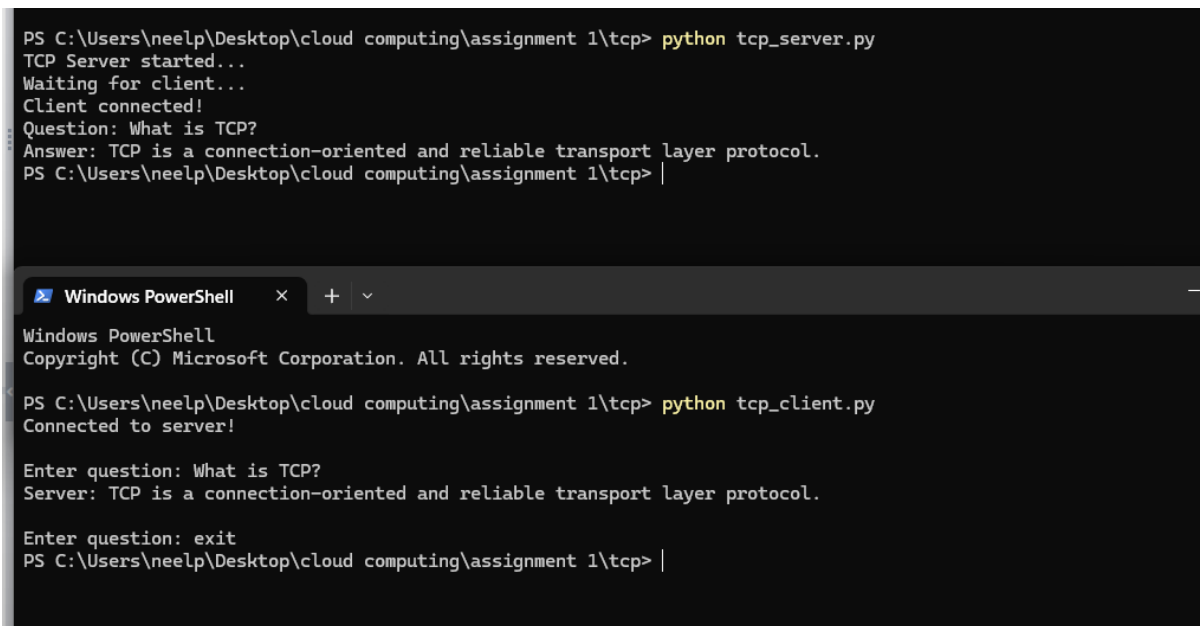
1 import socket
2
3 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4
5 client.connect(("127.0.0.1", 8080))
6
7 print("Connected to server!")
8
9 while True:
10
11     question = input("\nEnter question: ")
12
13     client.send(question.encode())
14
15     if question == "exit":
16         break
17
18     answer = client.recv(1024).decode()
19
20     print("Server:", answer)
21
22 client.close()

```

Output and Explanation

Explanation of Execution: When the server script starts, it listens on port 8080. Once the client connects, the user is prompted to enter a question. The client sends this question to the server over the reliable TCP connection. The server opens `qa.txt`, reads the file line-by-line, and splits the string using the `|` delimiter. If the received question matches a question in the file, it extracts the corresponding answer and sends it back. If no match is found, it sends "Question not found." Type "exit" terminates the loop and safely closes the connection on both ends.

Expected Output (Client Console):



```
PS C:\Users\neelp\Desktop\cloud computing\assignment 1\tcp> python tcp_server.py
TCP Server started...
Waiting for client...
Client connected!
Question: What is TCP?
Answer: TCP is a connection-oriented and reliable transport layer protocol.
PS C:\Users\neelp\Desktop\cloud computing\assignment 1\tcp> |

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\neelp\Desktop\cloud computing\assignment 1\tcp> python tcp_client.py
Connected to server!

Enter question: What is TCP?
Server: TCP is a connection-oriented and reliable transport layer protocol.

Enter question: exit
PS C:\Users\neelp\Desktop\cloud computing\assignment 1\tcp> |
```

Figure 1: Output for TCP communication

Exercise 2: UDP Client-Server with Checksum Error Detection

Problem Statement

Create a UDP server and client using socket programming. Implement a 32-bit checksum function over the data packet and include it in the header to detect errors, as UDP does not natively guarantee reliability. Introduce an artificial error to test if the server correctly drops corrupted packets.

Input Files

1. UDP Server (udp_server.py)

```
1 import socket
2
3 def calculateChecksum(data):
4     checksum = 0
5
6     for character in data:
7         checksum = (checksum + ord(character)) & 0xFFFFFFFF
8
9     return checksum
10
11
12 server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
13
14 server.bind(("127.0.0.1", 8081))
15
16 print("UDP Server started...")
17 print("Waiting for packets...")
18
19 while True:
20
21     data, address = server.recvfrom(1024)
22
23     packet = data.decode()
24
25     if packet == "exit":
26         break
27
28     parts = packet.split("|", 1)
29
30     if len(parts) != 2:
31         server.sendto(
32             "ERROR: Invalid packet".encode(),
33             address
34         )
35         continue
36
37     receivedChecksum = int(parts[0])
38     message = parts[1]
39
40     calculatedChecksum = calculateChecksum(message)
41
```

```

42 print("\nReceived message:", message)
43 print("Received checksum:", receivedChecksum)
44 print("Calculated checksum:", calculatedChecksum)
45
46 if receivedChecksum == calculatedChecksum:
47
48     print("Checksum matched.")
49     print("Packet accepted.")
50
51     response = "Packet accepted."
52
53 else:
54
55     print("Checksum mismatch!")
56     print("Packet rejected.")
57
58     response = "ERROR: Packet corrupted. Checksum mismatch."
59
60     server.sendto(response.encode(), address)
61
62 server.close()

```

2. UDP Client (udp_client.py)

```

1 import socket
2
3 def calculateChecksum(data):
4     checksum = 0
5
6     for character in data:
7         checksum = (checksum + ord(character)) & 0xFFFFFFFF
8
9     return checksum
10
11
12 client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
13
14 serverAddress = ("127.0.0.1", 8081)
15
16 while True:
17
18     message = input("\nEnter message: ")
19
20     if message == "exit":
21         client.sendto("exit".encode(), serverAddress)
22         break
23
24     checksum = calculateChecksum(message)
25
26     print("Checksum:", checksum)
27
28     packet = str(checksum) + "|" + message
29
30     choice = input("Introduce error? (y/n): ")
31
32     if choice.lower() == "y":
33

```

```

34     # Change the message after calculating checksum
35     if len(message) > 0:
36         message = "X" + message[1:]
37
38     packet = str(checksum) + "|" + message
39
40     print("Artificial error introduced!")
41
42     client.sendto(packet.encode(), serverAddress)
43
44     print("Packet sent.")
45
46     response, address = client.recvfrom(1024)
47
48     print("Server:", response.decode())
49
50 client.close()

```

Output and Explanation

Explanation of Execution: The UDP client prompts the user for a message and calculates a 32-bit checksum based on the characters' ASCII values using the `calculateChecksum` function. It creates a packet format of `checksum|message`. Before sending, the user is asked if they want to introduce an artificial error. If "y" is selected, the first character of the message payload is replaced with an 'X', but the checksum is **not** recalculated.

Upon receiving the datagram, the server splits the packet by the | delimiter. It extracts the client's calculated checksum and the message payload. The server then recalculates the checksum independently using the received payload. If the server's calculated checksum matches the received one (normal scenario), it accepts the packet. If they differ (error introduced scenario), it rejects the packet and sends an error message back to the client, effectively demonstrating error detection over an unreliable protocol.

Expected Output :

```

PS C:\Users\neelp\Desktop\cloud computing\assignment 1\udp> python udp_server.py
UDP Server started...
Waiting for packets...

Received message: Neel
Received checksum: 388
Calculated checksum: 388
Checksum matched.
Packet accepted.

Received message: Xeel
Received checksum: 424
Calculated checksum: 402
Checksum mismatch!
Packet rejected.

```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\neelp\Desktop\cloud computing\assignment 1\udp> python udp_client.py

Enter message: Neel
Checksum: 388
Introduce error? (y/n): n
Packet sent.
Server: Packet accepted.

Enter message: neil
Checksum: 424
Introduce error? (y/n): y
Artificial error introduced!
Packet sent.
Server: ERROR: Packet corrupted. Checksum mismatch.

Enter message: 

```

Figure 2: Output for UDP communication